

Algorithms

Lecture # 16

Syed Hamed Raza

Counting Sort: Stable

Counting Sort: Stable

Counting sort is stable. Stability is important because

- data are often carried with the keys being sorted.
- radix sort (which uses counting sort as a subroutine) relies on it to work correctly.

Counting Sort: Stable

Counting Sort: Stable

How is stability achieved?

COUNTING-SORT(array A, array B, int k)

1 $\vdots$

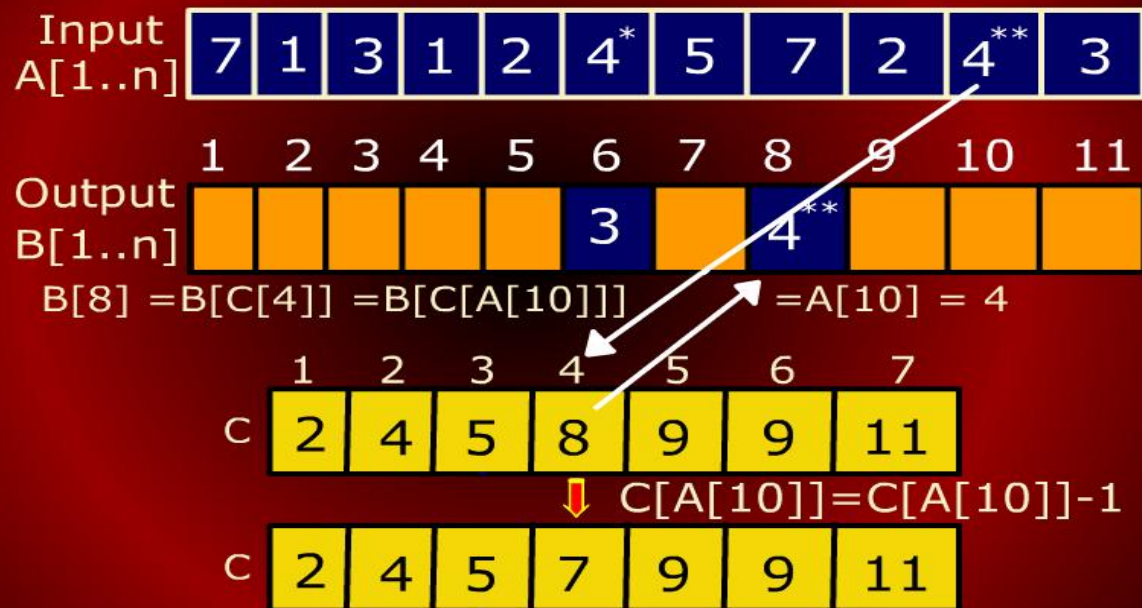
2 **for** $j \leftarrow \text{length}[A]$ **downto** 1

3 **do** $B[C[A[j]]] \leftarrow A[j]$

4 $C[A[j]] \leftarrow C[A[j]] - 1$

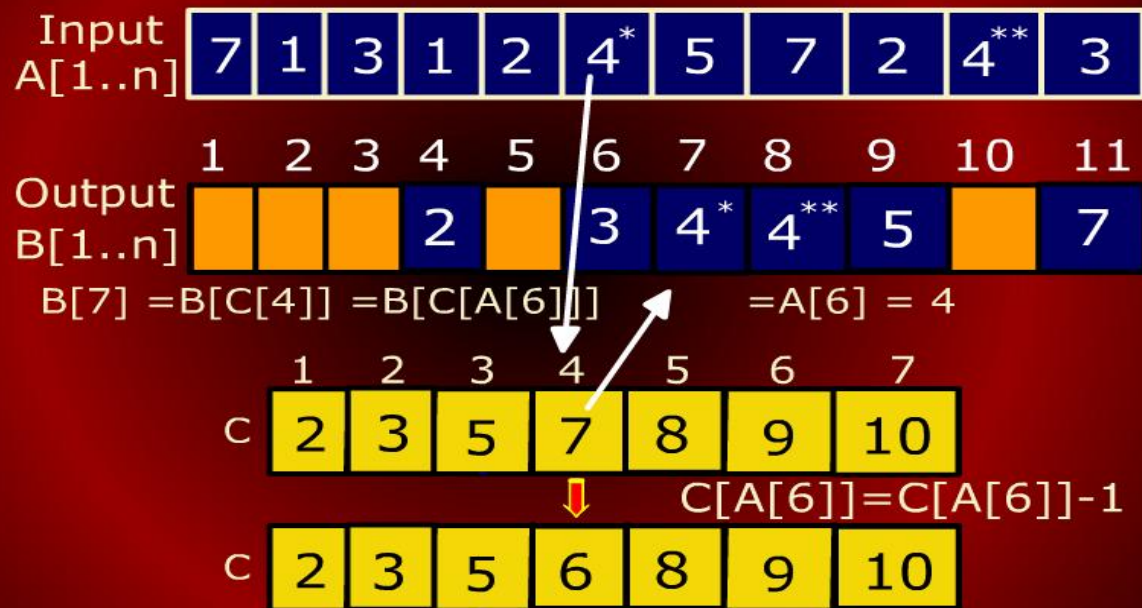
Counting Sort

Counting Sort



Counting Sort

Counting Sort



Counting Sort

Counting Sort

Input
A[1..n]

7'	1^	3#	1^^	2+	4*	5	7"	2++	4**	3##
1	2	3	4	5	6	7	8	9	10	11

Output
B[1..n]

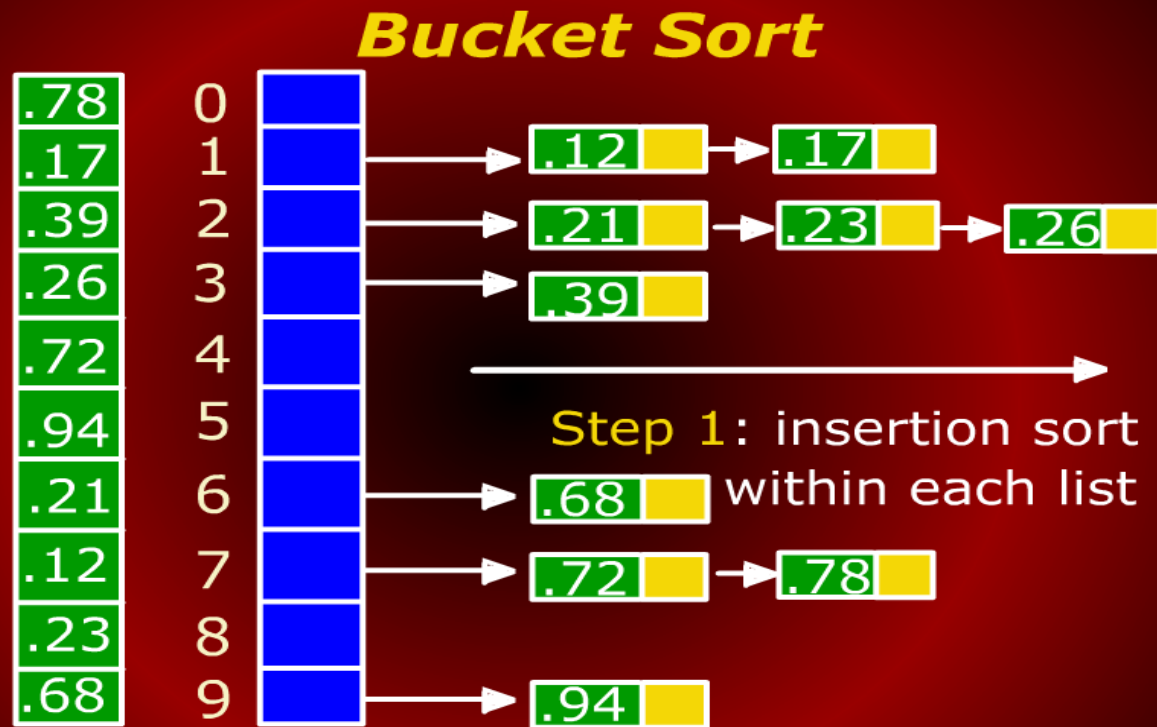
1^	1^^	2+	2++	3#	3##	4*	4**	5	7'	7"
----	-----	----	-----	----	-----	----	-----	---	----	----

Bucket Sort

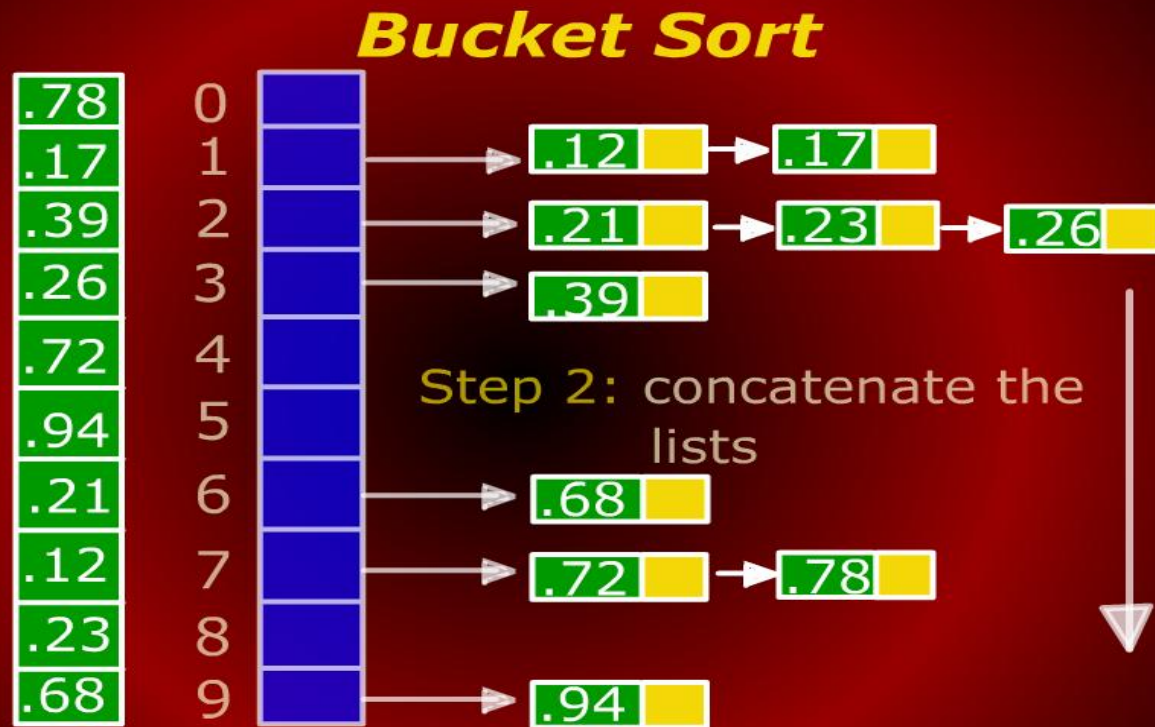
Bucket Sort

- Assumption: input elements are uniformly distributed over $[0, 1]$
- n inputs dropped into n equal-sized subintervals of $[0, 1]$.

Bucket Sort



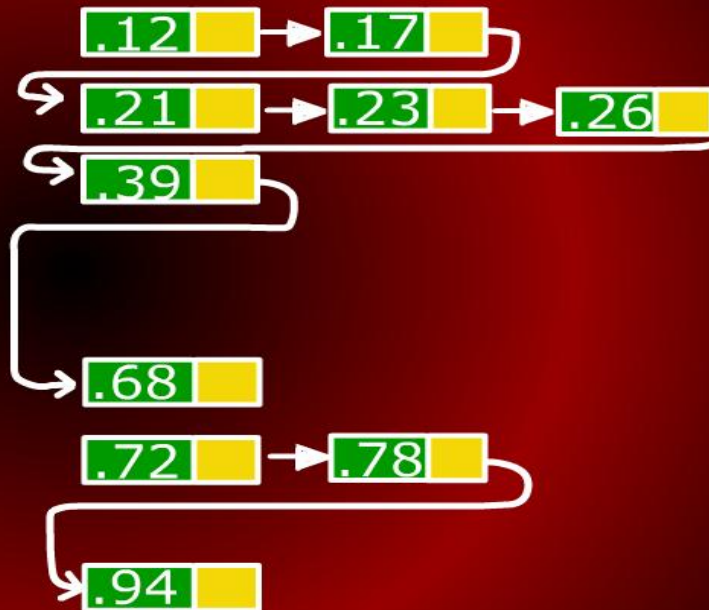
Bucket Sort



Bucket Sort

Bucket Sort

.78
.17
.39
.26
.72
.94
.21
.12
.23
.68



Radix Sort

Radix Sort

- The main shortcoming of counting sort is that it is useful for small integers, i.e., $1..k$ where k is small.
- If k were a million or more, the size of the rank array would also be a million.
- Radix sort provides a nice work around this limitation by sorting numbers one digit at a time.

Radix Sort

Radix Sort

576	49[4]	9[5]4	[1]76	176
494	19[4]	5[7]6	[1]94	194
194	95[4]	1[7]6	[2]78	278
296	$\Rightarrow$ 57[6]	$\Rightarrow$ 2[7]8	$\Rightarrow$ [2]96	$\Rightarrow$ 296
278	29[6]	4[9]4	[4]94	494
176	17[6]	1[9]4	[5]76	576
954	27[8]	2[9]6	[9]54	954

Radix Sort

Radix Sort

Here is the algorithm that sorts $A[1..n]$ where each number is d digits long.

RADIX-SORT(array A , int n , int d)

```
1  for  $i \leftarrow 1$  to  $d$ 
2  do stably sort  $A$  w.r.t  $i^{\text{th}}$  lowest
   order digit
```

Dynamic Programming

Dynamic Programming

Motivation

Motivation

- Suppose we put a pair of rabbits in a place surrounded on all sides by a wall.
- How many pairs of rabbits can be produced from that pair in a year if it is supposed that every month each pair begets a new pair which from the second month on becomes productive?
- Resulting sequence is
1, 1, 2, 3, 5, 8, 13, 21, 34, 55, . . .
- Each number is the sum of the two preceding numbers.

Leonardo Fibonacci Pisano

Leonardo Fibonacci Pisano

- This problem was posed by Leonardo Pisano, better known by his nickname Fibonacci (son of Bonacci, born 1170, died 1250).
- This problem and many others were in posed in his book *Liber abaci*, published in 1202.
- The book was based on the arithmetic and algebra that Fibonacci had accumulated during his travels.

Leonardo Fibonacci Pisano

Leonardo Fibonacci Pisano

- The book, which went on to be widely copied and imitated, introduced the Hindu-Arabic place-valued decimal system and the use of Arabic numerals into Europe.
- The rabbits problem in the third section of Liber abaci led to the introduction of the Fibonacci numbers and the Fibonacci sequence for which Fibonacci is best remembered today.

Leonardo Fibonacci Pisano

Leonardo Fibonacci Pisano

- This sequence, in which each number is the sum of the two preceding numbers, has proved extremely fruitful and appears in many different areas of mathematics and science.
- The *Fibonacci Quarterly* is a modern journal devoted to studying mathematics related to this sequence.

Fibonacci Sequence

Fibonacci Sequence

- The Fibonacci numbers F_n are defined as follows:

$$F_0 = 0$$

$$F_1 = 1$$

$$F_n = F_{n-1} + F_{n-2}$$

Fibonacci Number

Fibonacci Number

The recursive definition of Fibonacci numbers gives us a recursive algorithm for computing them:

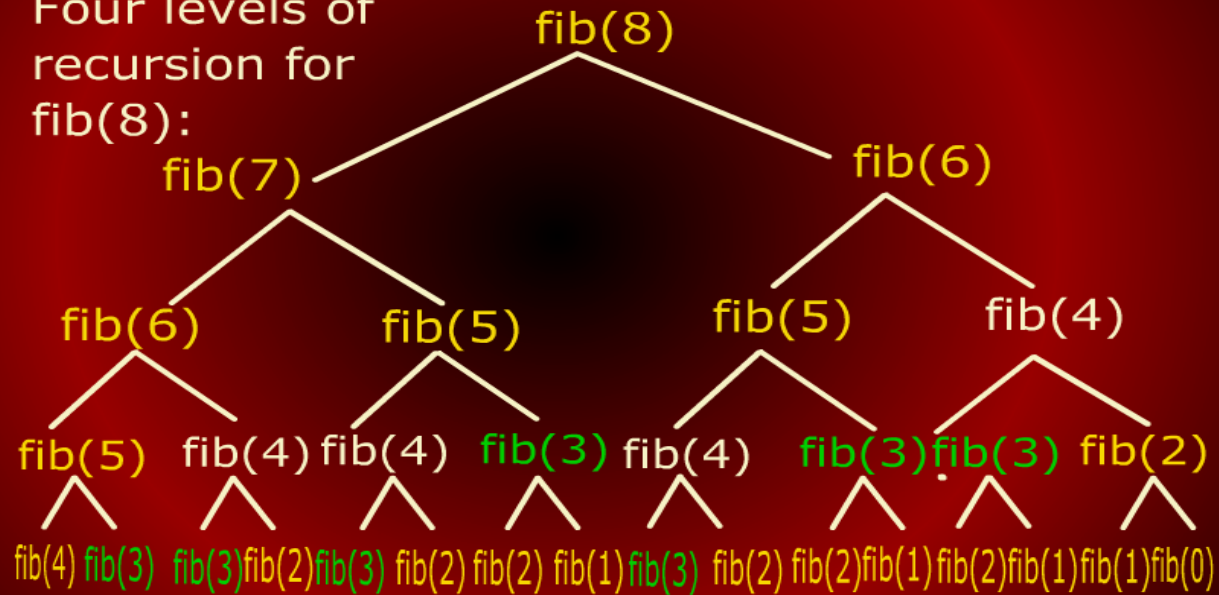
FIB(n)

```
1  if ( $n < 2$ )  
2  then return  $n$   
3  else return FIB( $n - 1$ ) + FIB( $n - 2$ )
```

Fibonacci Number: Recursive Calls

Fibonacci Number: Recursive Calls

Four levels of
recursion for
fib(8):



Fibonacci Number: Recursive Calls

Fibonacci Number: Recursive Calls

- A single recursive call to `fib(n)` results in one recursive call to `fib(n - 1)`, two recursive calls to `fib(n - 2)`, three recursive calls to `fib(n - 3)`, five recursive calls to `fib(n - 4)` and, in general, F_{k-1} recursive calls to `fib(n - k)`
- For each call, we're recomputing the same fibonacci number from scratch.

Fibonacci Number: Recursive Calls

Fibonacci Number: Recursive Calls

- We can avoid this unnecessary repetitions by writing down the results of recursive calls and looking them up again if we need them later.
- This process is called *memoization*.

Fibonacci Number: Memoization

Fibonacci Number: Memoization

```
MEMOFIB(n)
1  if (n < 2)
2      then return n
3  if (F[n] is undefined)
4      then F[n] ← MEMOFIB(n - 1)
                   + MEMOFIB(n - 2)
5  return F[n]
```

Fibonacci Number: Recursive Calls

Fibonacci Number: Recursive Calls

- If we trace through the recursive calls to MEMOFIB, we find that array F[] gets filled from bottom up.

```
MEMOFIB(n)
1 if (n < 2)
2   then return n
3 if(F[n] is undefined)
4   then F[n] ←
        MEMOFIB(n - 1)
        +MEMOFIB(n - 2)
5   return F[n]
```

Fibonacci Number: Recursive Calls

Fibonacci Number: Recursive Calls

to MEMOFIB, we find that array F[] gets filled from bottom up.

- i.e., first F[2], then F[3], and so on, up to F[n].

```
MEMOFIB(n)
1 if (n < 2)
2   then return n
3 if(F[n] is undefined)
4   then F[n] ←
        MEMOFIB(n - 1)
        +MEMOFIB(n - 2)
5
6 return F[n]
```

Fibonacci Number: Recursive Calls

Fibonacci Number: Recursive Calls

- We can replace recursion with a simple for-loop that just fills up the array $F[]$ in that order.

```
MEMOFIB(n)
1 if (n < 2)
2   then return n
3 if(F[n] is undefined)
4   then F[n] ←
           MEMOFIB(n - 1)
           +MEMOFIB(n - 2)
6 return F[n]
```

Fibonacci Number: Iterative Algorithm

Fibonacci Number: Iterative Algorithm

This gives us our first explicit dynamic programming algorithm.

ITERFIB(n)

```
1    $F[0] \leftarrow 0$ 
2    $F[1] \leftarrow 1$ 
3   for  $i \leftarrow 2$  to  $n$ 
4   do
5        $F[i] \leftarrow F[i - 1] + F[i - 2]$ 
6   return  $F[n]$ 
```


Counting Sort

Radix Sort

Fibonacci Sequence

$$F_n = F_{n+1} + F_{n+2}$$

